



Deadlines for DAGs: What's Shipped and What's Next

Airflow Summit · Austin, TX · Aug 31, 2026

Sean Ghaeli

Software Development Engineer · AWS



Your pipeline was late. When did you find out?

The 9am dashboard was **empty**.

The pipeline finished at **11**. Nobody was paged.

Late data discovered **after the fact** is the default failure mode of orchestration.

Legacy SLAs

SLAs: set a maximum expected duration for a run, get an email when it was exceeded.

What legacy SLA gave you

- Evaluated inside the scheduler loop
- Notification was the only possible response

What you actually needed

- Run-level lateness against **meaningful** reference points
- Callbacks that can be programmed to **do anything** in response

Deadline Alerts (AIP-86)



reference point + **interval** + **callback**

```
from datetime import timedelta
from airflow.sdk import (DAG, AsyncCallback,
    DeadlineAlert, DeadlineReference)
```

```
with DAG(
    deadline=DeadlineAlert(
        reference=DeadlineReference.DAGRUN_LOGICAL_DATE,
        interval=timedelta(hours=7),
        callback=AsyncCallback(notify_oncall),
    )
```

What It Isn't

- | **Not a timeout.** It never kills anything; the run keeps going.
- | **Not a retry policy.** Retries are about failure; deadlines are about lateness.
- | **Not just email.** The callback is arbitrary code with real platform access.

Dynamic Intervals: VariableInterval

```
from airflow.sdk.definitions.deadline import VariableInterval

DeadlineAlert(
    name="ops-tunable",
    reference=DeadlineReference.DAGRUN_QUEUED_AT,
    interval=VariableInterval("nightly_finance_deadline"),
    callback=SyncCallback(notify_oncall),
)
```

```
airflow variables set nightly_finance_deadline 5400
```

The Variable holds seconds. Change it from the CLI, UI or API; no redeploy.

Seeing Deadlines in the UI



Home

Dags

Assets

Browse

Admin

Docs

06:00

Dag

Dead

Options

before

Dag Run

scheduled__2026-04-20T13:56:31.693633+00:00

Filter

00:00:14.830

00:00:07.315

start

fail

Task Instances

Asset Events

Audit Log

Code

Details

Filter

Task ID

Map Index

State

Start Date

Try Number

Operator

Duration

fail

1

Failed

2026-04-20 07:56:33

1

BashOperator

00:00:00.093

start

1

Success

2026-04-20 07:56:32

1

BashOperator

00:00:00.121

search

Search Dags

✖+K

Trigger

scheduled__2026-04-20T13:56:31.693633+00:00

✖ Failed

Logical Date

Run Type

Start Date

End Date

Duration

Dag Version(s)

Deadline

2026-04-20 07:56:31

⌚ Scheduled

2026-04-20 07:56:32

2026-04-20 07:56:34

00:00:02.208

v1

⚠ Missed

(1-minute alert)

Must complete within a minute of queue time

Expected: 2026-04-20 07:57:32

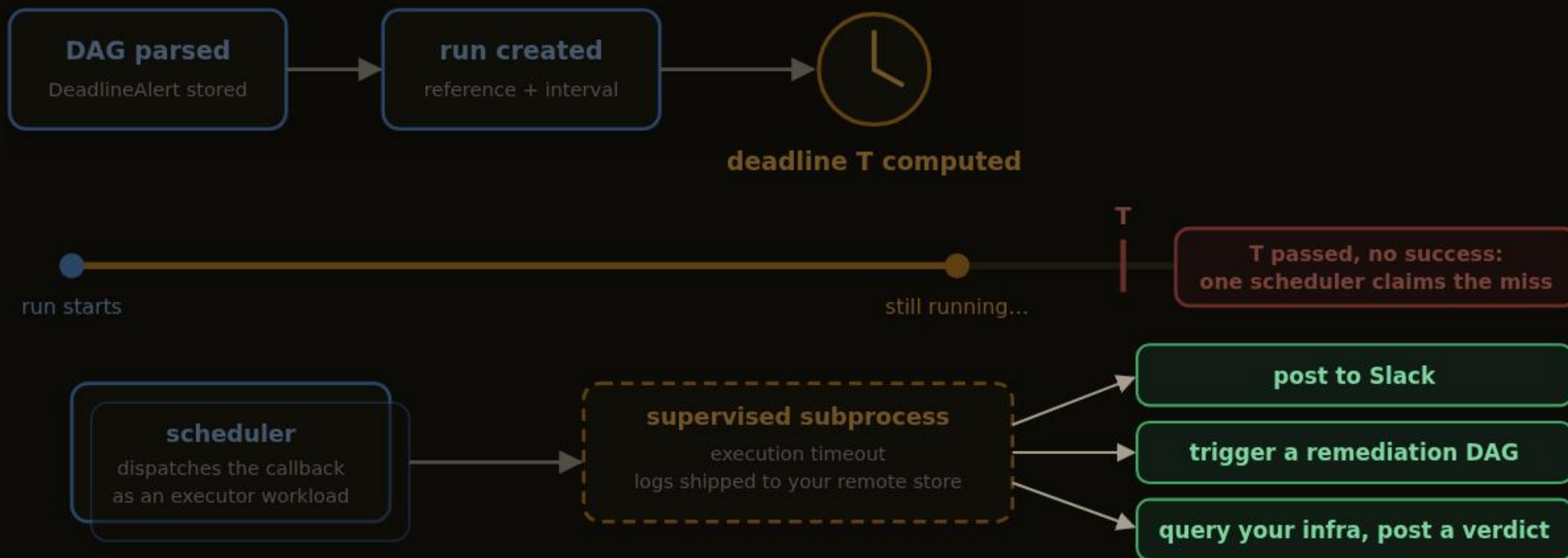
Actual: 2026-04-20 07:56:34

Finished 00:00:57 before deadline

Dag Run page: deadline status on the run header

7 / 16

How a Deadline Fires



The action: it is code, it can do anything. The callback runs with platform access and the platform watching it: post to Slack, trigger a DAG, query your infra.

The Callback Contract

Supervised. A subprocess with an execution timeout and remote logging; a hung callback can't wedge anything.

Reliable. One callback fires per miss, even with multiple scheduler replicas running.

Empowered. Reads Connections, Variables and Assets through the supervisor channel, with secrets masked. The callback can take real action.

Engine internals: **Ramit Kataria**'s talk, "Airflow Callbacks Revamped".

Example 1: Notify with Context

A deadline callback on the **nightly_finance** DAG, wired to an alert named **finance-close**.

```
def notify_with_context(*, alert, context, **_):  
    run = context["dag_run"]  
    hook = SlackWebhookHook(  
        slack_webhook_conn_id="slack_alerts")  
    hook.send(text=(  
        f":alarm_clock: deadline '{alert}' "  
        f"missed for {run['dag_id']} "  
        f"run {run['dag_run_id']}"  
    ))
```

Deadline Bot APP 9:01 AM

● deadline 'finance-close' missed for **nightly_finance** run
scheduled__2026-08-25T02:00

Example 2: Trigger Remediation

```
def remediate(*, alert, context, **kwargs):  
    notify_with_context(alert=alert,  
                        context=context, **kwargs)  
  
    run_id = context["dag_run"]["dag_run_id"]  
    client.trigger_dag_run(  
        dag_id="finance_catchup",  
        conf={"source_run": run_id},  
    )
```



the human gets the message
and the remediation is already running

Example 3: Post a Diagnosis

```
def diagnose(*, alert, **_):  
    wh = PostgresHook(  
        postgres_conn_id="warehouse")  
    n = wh.get_first(  
        "SELECT count(*) "  
        "FROM staging.orders_today")[0]  
  
    verdict = (  
        "upstream staging.orders_today is EMPTY"  
        if n == 0 else  
        "upstream OK; run is just slow")  
    post_to_slack(  
        f"'{alert}' missed. "  
        f"Likely cause: {verdict}")
```

Deadline Bot APP 9:01 AM

● 'finance-close' missed.

Likely cause: upstream staging.orders_today is EMPTY

The callback reads live state at fire time.

The Timeline

Airflow 3.1

● shipped

Deadline Alerts arrive

Airflow 3.4

● awaiting release

Supervised callbacks

Variable intervals

Named alerts

UI pages

Calendar view

Fresh context fetch

Beyond

○ design stage

Task-level deadlines

Asset deadlines

● shipped ● awaiting release ○ design stage

looking further out

• alarm / ◦ reflex / ◦ anticipation

From Reactive to Proactive

Today

Deadlines are reactive: they act after something has already gone wrong.

The question

What if they could be proactive?

looking further out

• alarm / • reflex / • anticipation

The Early-Warning Deadline

Set a second deadline earlier than the real one. When it fires, there is still time to act.



The callback effectively says: "I was going to be late, so I scaled the workers."

Thank You



This session
slides and links



Get involved
area:deadline-alerts on GitHub

Go deeper: **Ramit Kataria**, "Airflow Callbacks Revamped: Beyond the Dag Processor" · **Dennis Ferruzzi**, "Advanced Deadline Alerts: Writing your own custom References and Callbacks"
Sean Ghaeli · AWS MWAA